Valid Anagram

Solved ✓

Easy | Topics | Company Tags | Hints

Given two strings `s` and `t`, return `true` if the two strings are anagrams of each other, otherwise return `false`.

An **anagram** is a string that contains the exact same characters as another string, but the order of the characters can be different.

Example 1:

```
Input: s = "racecar", t = "carrace"

Output: true
```

Example 2:

```
Input: s = "jar", t = "jam"

Output: false
```

Constraints:

Solution 1 +

NeetBot | Hint | Ln 1, Col 1

```java
class Solution {
    public boolean isAnagram(String s, String t) {
 if (s.length() != t.length()) {
            return false;
        }

        int[] count = new int[26];

        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }

        for (int c : count) {
            if (c != 0) {
                return false;
            }
        }

        return true;
    }
}
```

Console ^ Run Submit

Replace Elements With Greatest Element On Right Side

Solved

Easy | Topics | Company Tags

You are given an array `arr`, replace every element in that array with the greatest element among the elements to its right, and replace the last element with `-1`.

After doing so, return the array.

Example 1:

```
Input: arr = [2,4,5,3,1,2]

Output: [5,5,3,2,2,-1]
```

Example 2:

```
Input: arr = [3,3]

Output: [3,-1]
```

Solution 1 +

NeetBot | Hint | Ln 1, Col 1

```java
class Solution {
    public int[] replaceElements(int[] arr) {
        int n = arr.length;
        int max = -1;

        for(int i = n - 1; i >= 0; i--) {
            int current = arr[i];
            arr[i] = max;
            if(current > max) {
                max = current;
            }
        }

        return arr;
    }
}
```

Click to collapse the range.

Console

Run | Submit

Is Subsequence

Solved ✓

Easy | Topics | Company Tags

You are given two strings `s` and `t`, return `true` if `s` is a **subsequence** of `t`, or `false` otherwise.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"abcde"` while `"aec"` is not).

Example 1:

```
Input: s = "node", t = "neetcode"

Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"

Output: false
```

```java
class Solution {
    public boolean isSubsequence(String s, String t) {
        int i = 0, j = 0;

        while (i < s.length() && j < t.length()) {
            if (s.charAt(i) == t.charAt(j)) {
                i++;
            }
            j++;
        }

        return i == s.length();
    }
}
```

Console ^ Run Submit